middleware, and as mentioned in chapter 4, you typically want to place your routing middleware after the static file middleware. As the routing middleware needs to move, so does the authorization middleware!

Traditionally, the authentication middleware is also placed after the routing middleware, before the authorization middleware, but this isn't crucial. The only requirement is that it's placed before any middleware that requires an authenticated user, such as the authorization middleware.

If you wish, you can move the location of the authentication middleware by calling `UseAuthentication()` at the appropriate point. I prefer to be explicit where possible, so I typically take this approach, moving it between the call to `UseRouting()` and `UseAuthorization()`:

```
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapRazorPages();
app.Run();
```

If you don't place the authentication middleware at the correct point in the pipeline, you can run into strange bugs where users aren't authenticated correctly or authorization policies aren't applied correctly. The templates work out of the box, but you need to take care if you're working with an existing application or moving middleware around.

Now that you've got an overview of the additions made by Identity, we'll look in a bit more detail at the database schema and how Identity stores users in the database.

23.3.3 The ASP.NET Core Identity data model

Out of the box, and in the default templates, Identity uses EF Core to store user accounts. It provides a base `DbContext` that you can inherit from, called `IdentityDbContext`, which uses an `IdentityUser` as the user entity for your application.

In the template, the app's `DbContext` is called `ApplicationDbContext`. If you open this file, you'll see it's sparse; it inherits from the `IdentityDbContext` base class I described earlier, and